

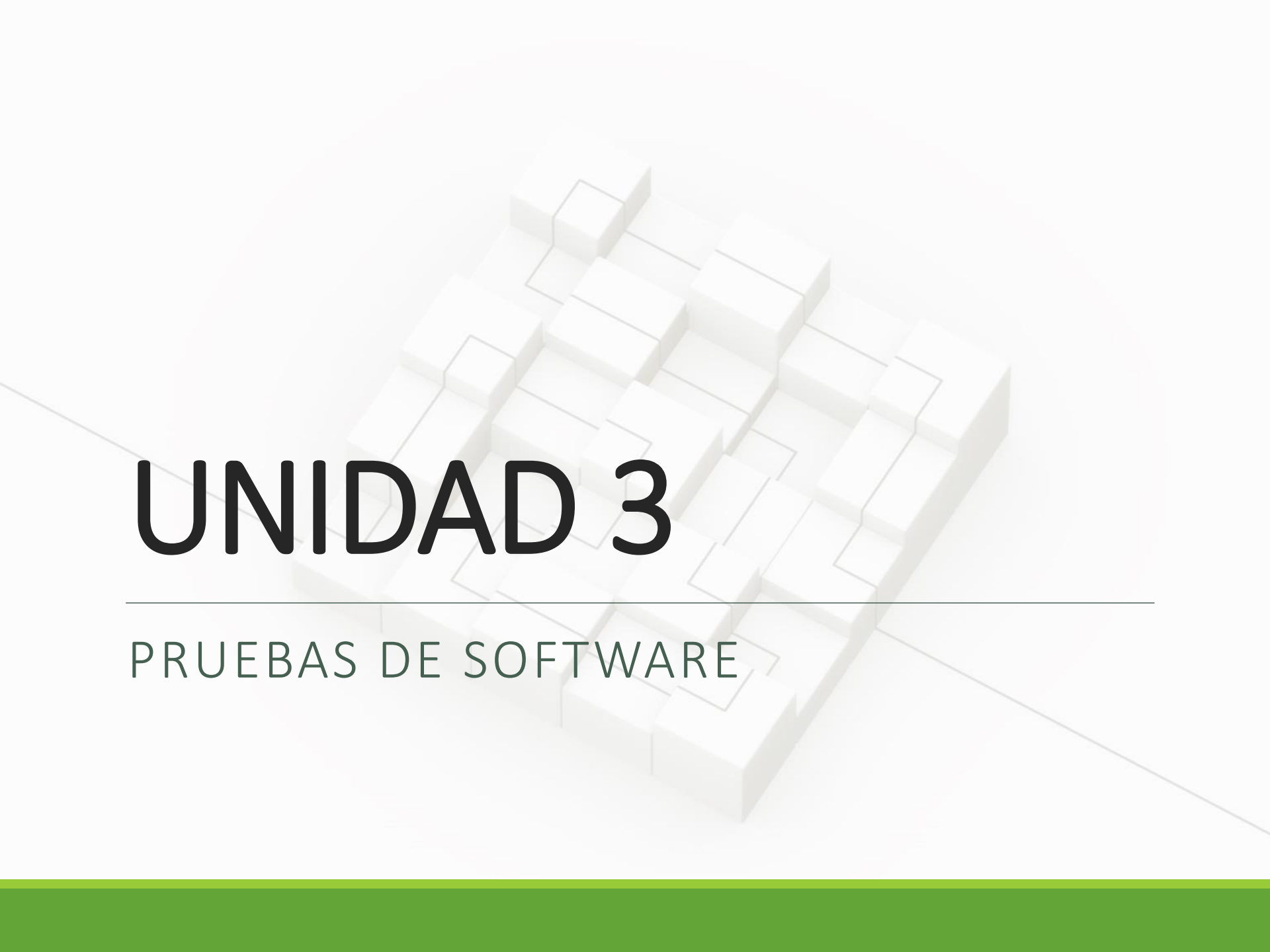


ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Programación Avanzada

LUIS CASTILLO



UNIDAD 3

PRUEBAS DE SOFTWARE

Entornos de Desarrollo de Pruebas

PRUEBAS DE SOFTWARE

A solid green horizontal bar at the bottom of the slide.

Plataformas colaborativas

CONTROL DE VERSIONES E INTEGRACIÓN CON
ENTORNOS

Fundamentos de las Plataformas Colaborativas

Control de Versiones Distribuido

Sistemas como Git que rastrean y gestionan los cambios en el código fuente.

Trazabilidad

Mantiene un registro detallado de todos los cambios y quién los realizó.

Validación de Integración

Asegura que los cambios se prueben y validen antes de fusionarse en el código principal.



Trabajo Simultáneo

Permite a múltiples desarrolladores trabajar en el mismo código al mismo tiempo.

Control de Cambios

Implementa procesos para gestionar y aprobar las modificaciones del código.

Orquestación del Ciclo de Vida del Software



Desarrollo

El proceso de escribir y refinar el código.



Pruebas

La verificación de la funcionalidad y la calidad del software.



Revisión

La evaluación del código por parte de otros desarrolladores.



Automatización

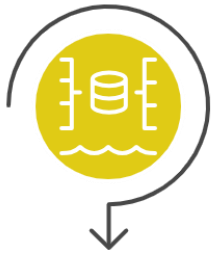
La implementación de procesos automatizados para la eficiencia.



Despliegue

La entrega del software a los usuarios finales.

Componentes fundamentales de Git



Repositorio

Almacén central del código y su historial.



Commits

Unidades atómicas de cambio en el repositorio.



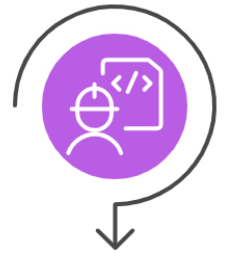
Ramas

Líneas paralelas de desarrollo en el repositorio.



Pull Requests

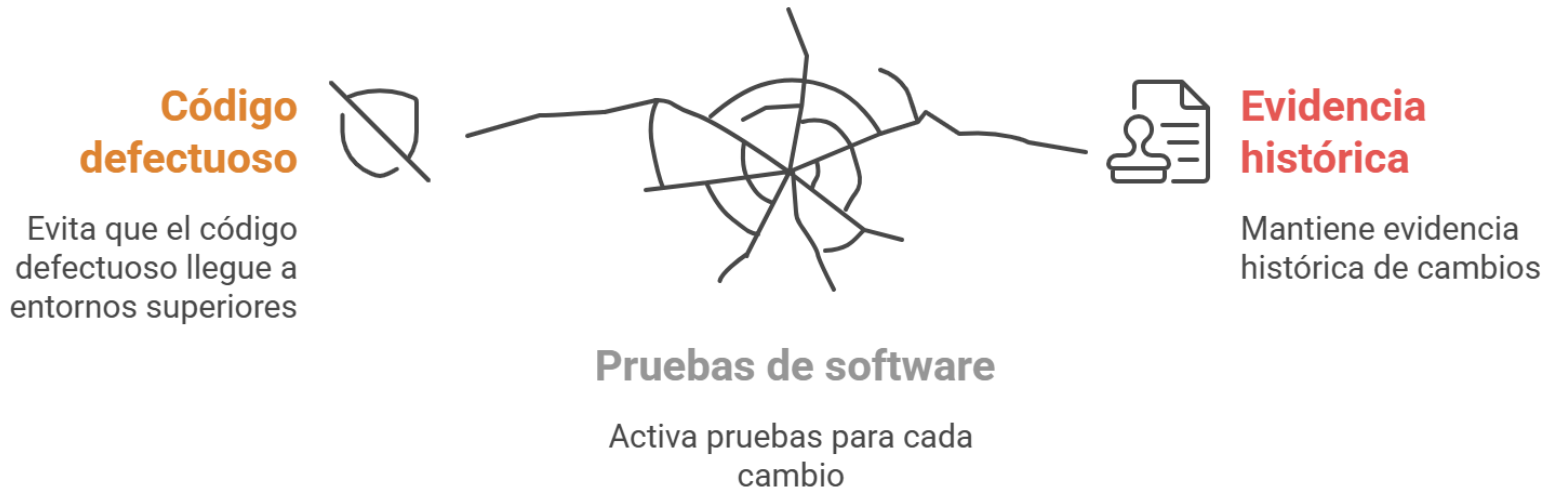
Mecanismo formal de integración de cambios.



Pipelines CI/CD

Automatización de pruebas y builds en el repositorio.

Las pruebas de software impactan el ciclo de vida del código



Las plataformas colaborativas
convierten las pruebas en un
requisito del proceso, no en
una actividad opcional

¿Qué plataforma de control de versiones se adapta mejor a tus necesidades?



GitHub

Ideal para comunidades abiertas y ecosistemas extensos



Bitbucket

Se integra con herramientas empresariales y flujos de trabajo

Flujo de trabajo en ramas

CONTROL DE VERSIONES E INTEGRACIÓN CON
ENTORNOS

Revelando el Flujo de Trabajo en Ramas

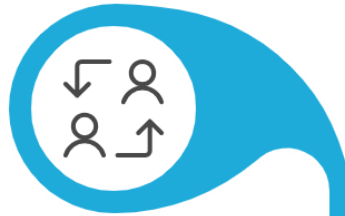
Estabilidad del Código Base

Mantiene el código base estable y confiable.



Aislamiento de Cambios

Separa cambios para evitar conflictos y errores.



Reducción de Errores

Minimiza errores al trabajar en ramas aisladas.



Tipos de ramas



main/master

Código listo para producción.

Integración de funcionalidades.

develop



feature/

Nuevas características.

Correcciones urgentes.

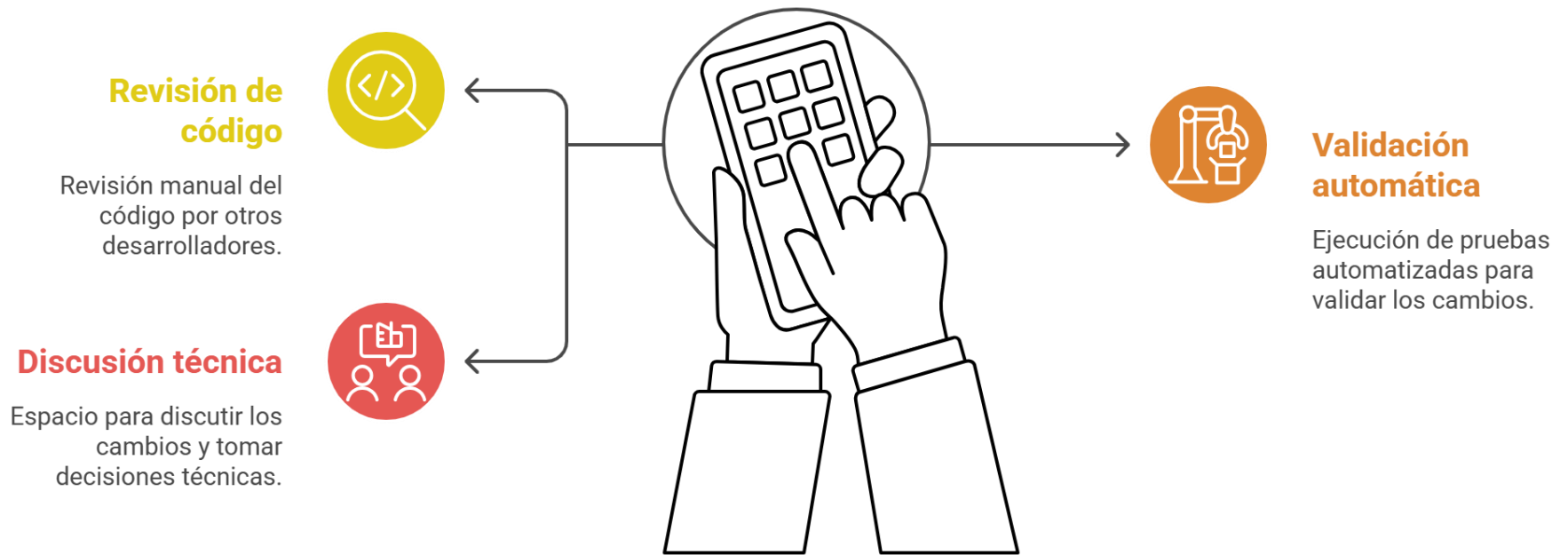
hotfix/



La fusión de ramas impacta el código



Componentes de una Pull Request



Relación ramas-entornos

Código desplegado

El código desplegado se encuentra en producción.



Código validado

El código validado se encuentra en la rama main.



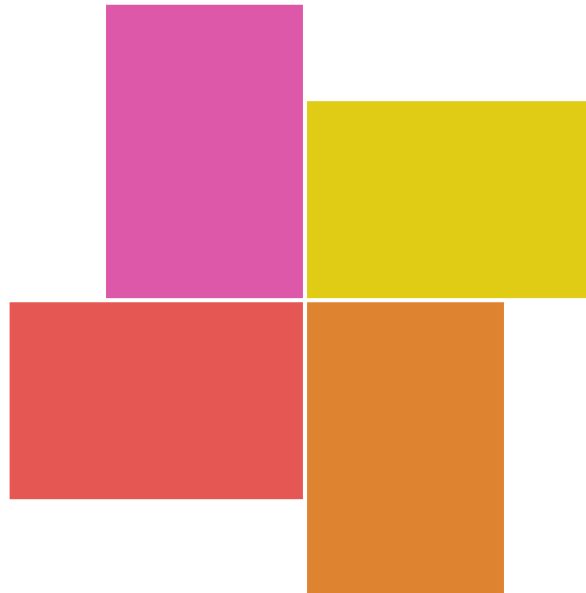
Código local

El código local se encuentra en la rama feature.



Código probado

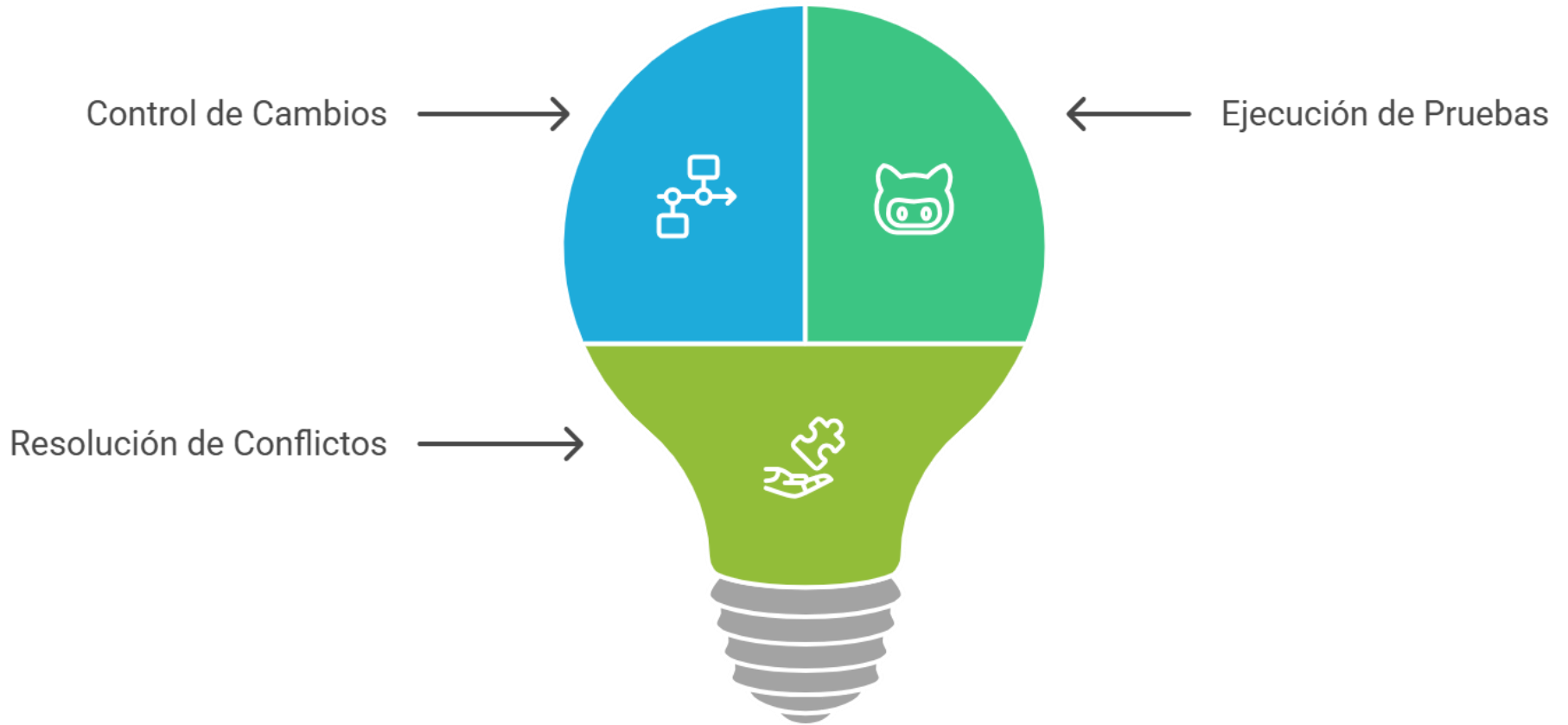
El código probado se encuentra en la rama develop.



Integración de control de versiones con IDEs

CONTROL DE VERSIONES E INTEGRACIÓN CON ENTORNOS

Beneficios de la Integración Git-IDE



Beneficios en pruebas

Pruebas antes del commit

El desarrollador puede ejecutar pruebas antes del commit.

Detección temprana de errores

Se detectan errores temprano.

Reducción de fallos en integración

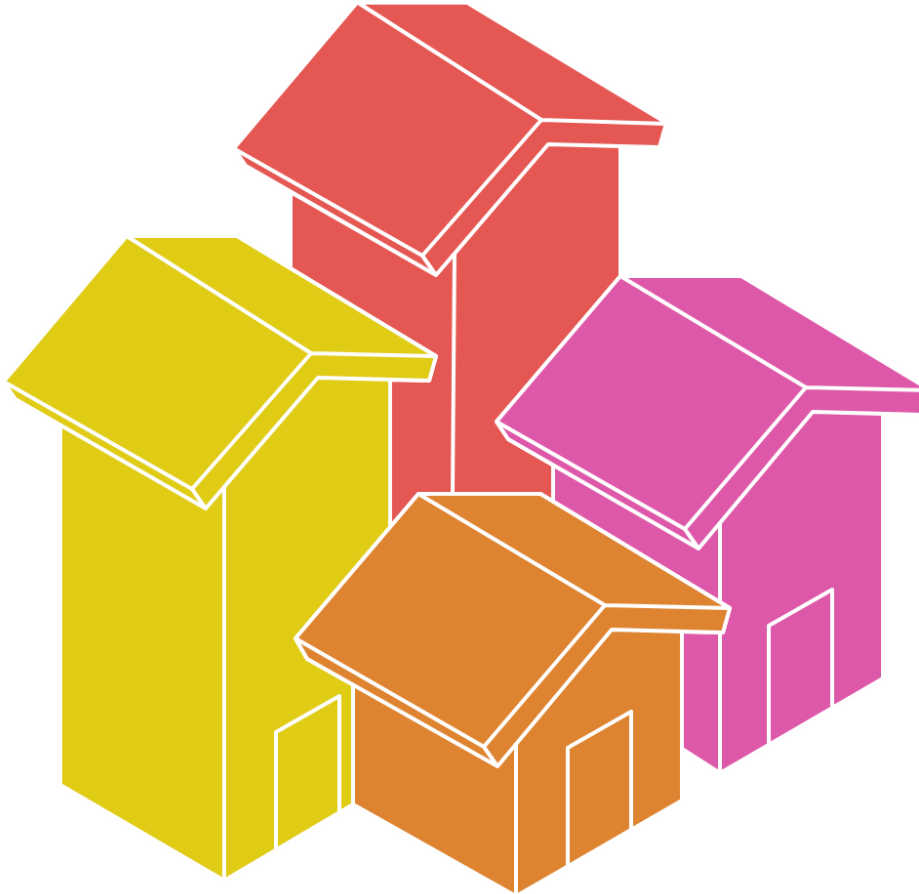
Se reduce la probabilidad de fallos en integración.

Herramientas de build

AUTOMATIZACIÓN Y ENTORNOS DE CONSTRUCCIÓN

A solid green horizontal bar at the bottom of the slide.

Herramientas de build



1

Compilación

Automatiza el proceso de compilación del código.

2

Ejecución de pruebas

Automatiza la ejecución de pruebas de software.

3

Empaquetado

Automatiza el proceso de empaquetado del software.

4

Generación de artefactos

Automatiza la generación de artefactos del software.

¿Cómo asegurar que el procedimiento de desarrollo sea efectivo?



Características de Maven



Basado en convenciones

Maven se basa en convenciones para simplificar la configuración del proyecto.



Ciclo de vida bien definido

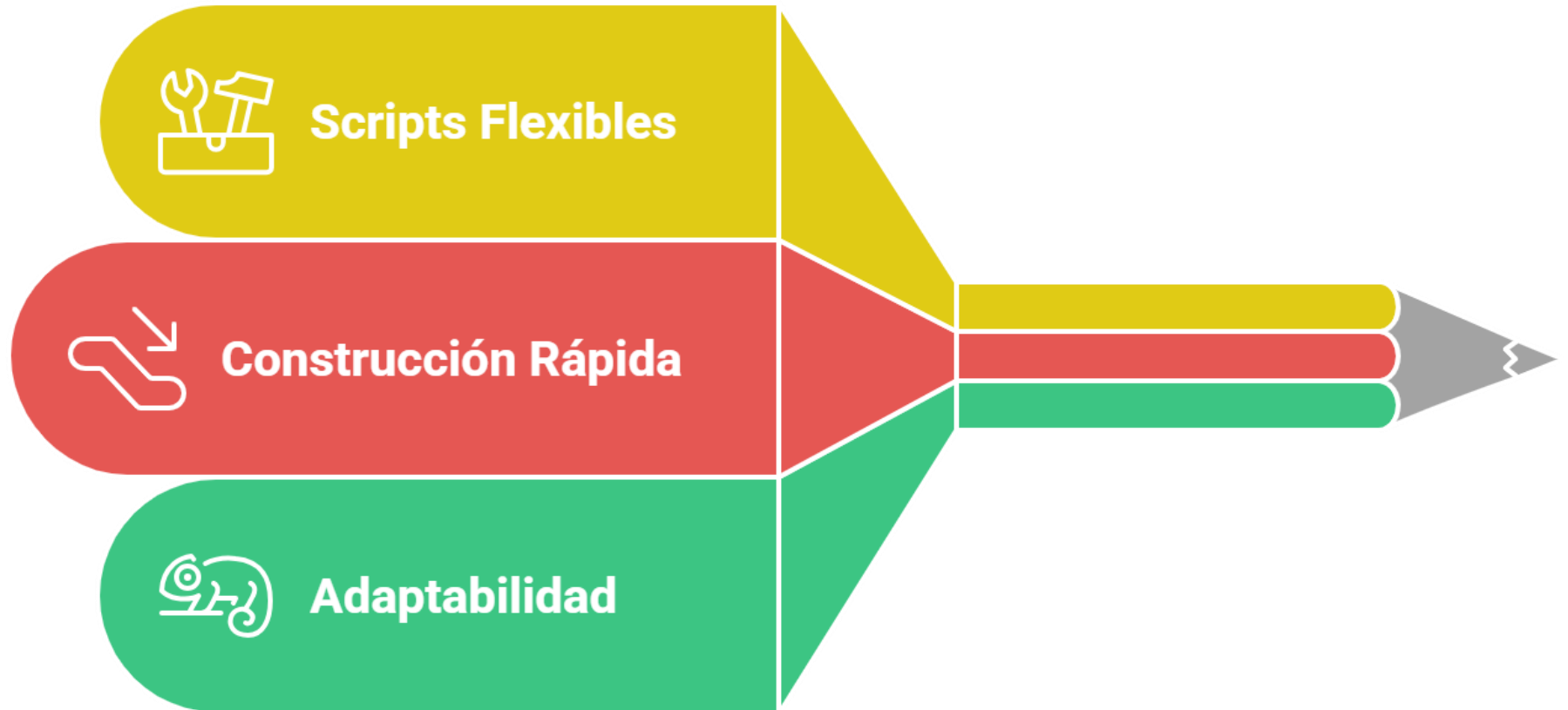
Maven utiliza un ciclo de vida bien definido con fases como compile, test y package.



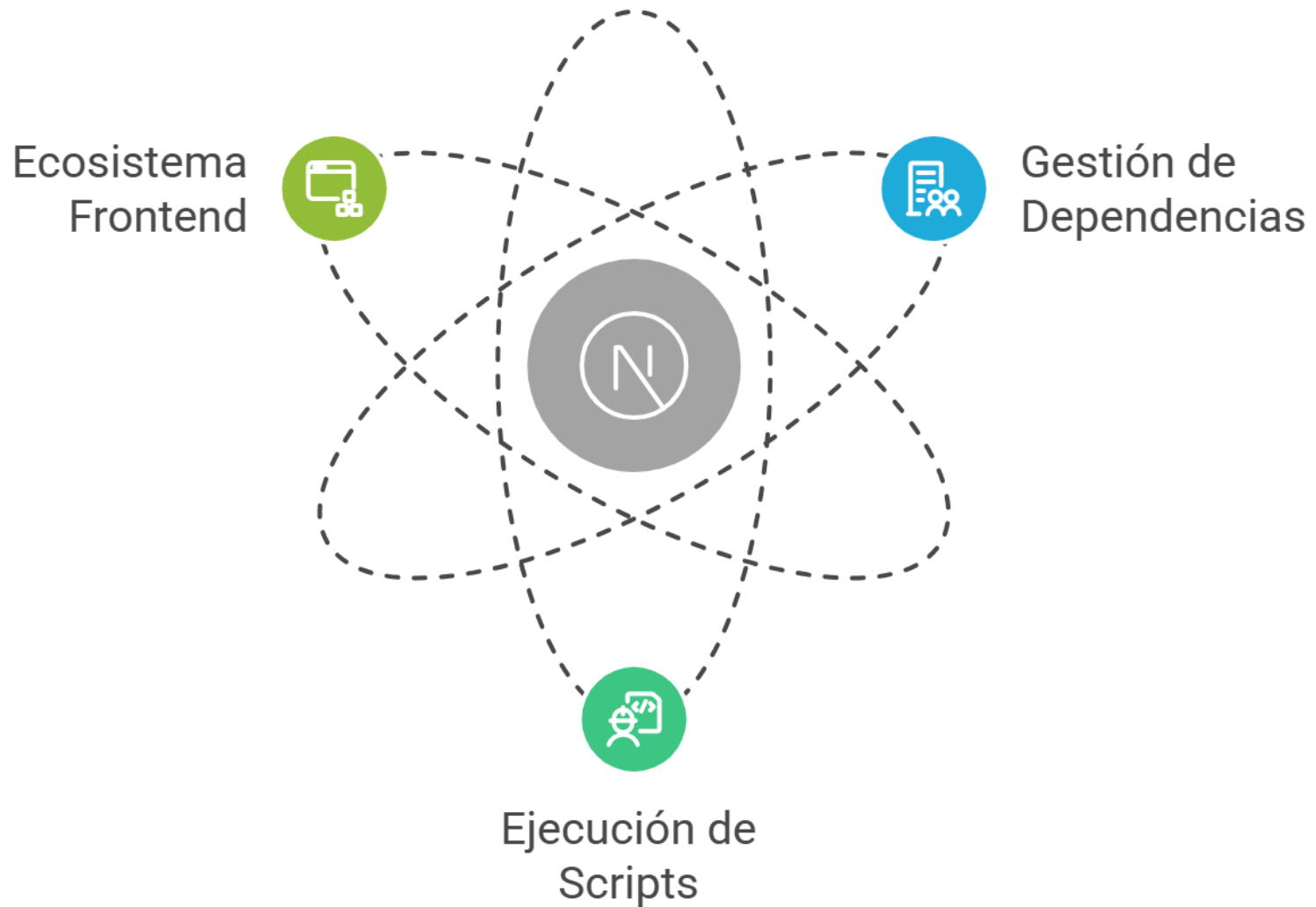
Ideal para proyectos estructurados

Maven es ideal para proyectos estructurados y estables, proporcionando una gestión de dependencias robusta.

Gradle en Proyectos Modernos



npm: Herramienta Integral de Desarrollo

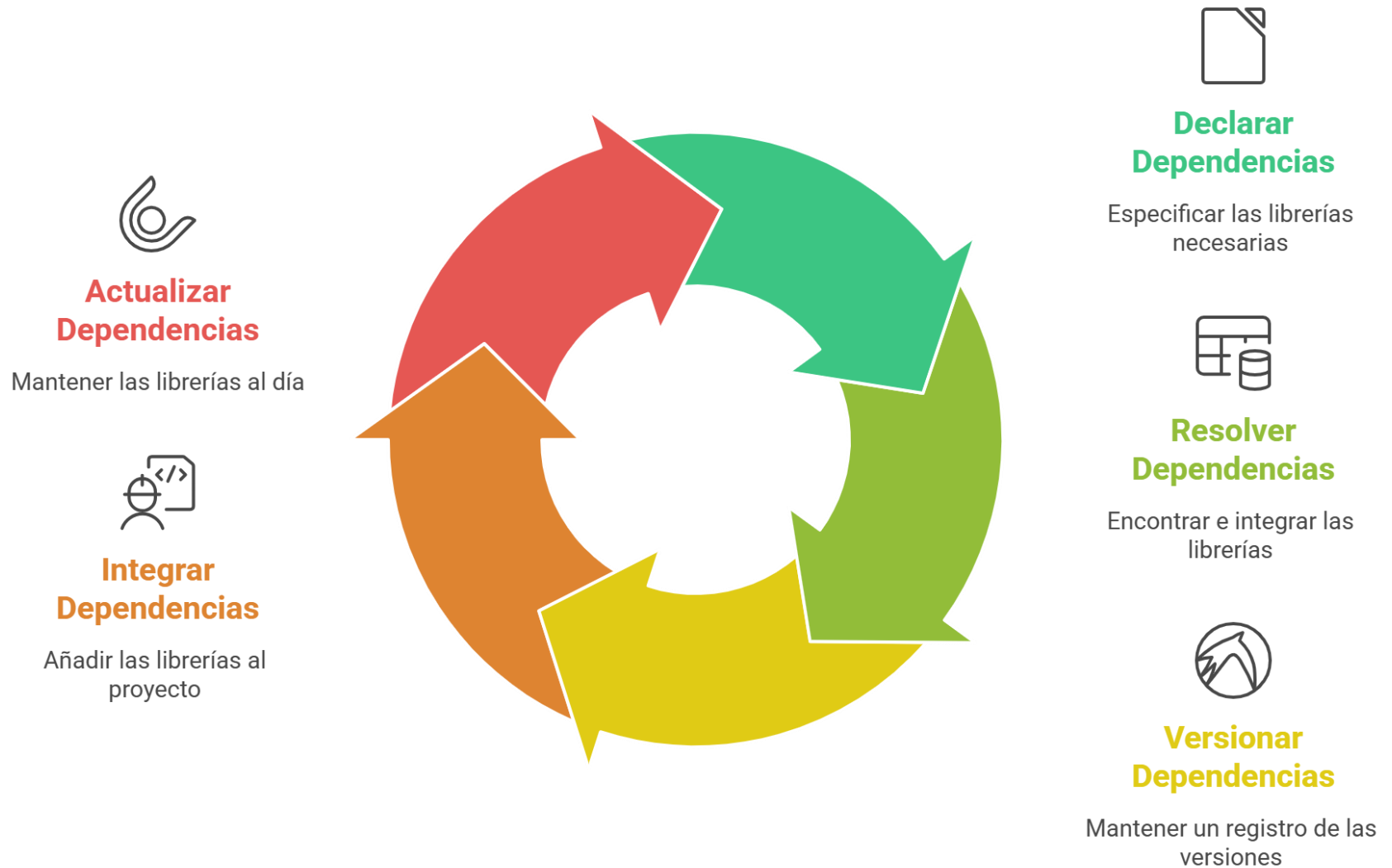


Gestión de dependencias

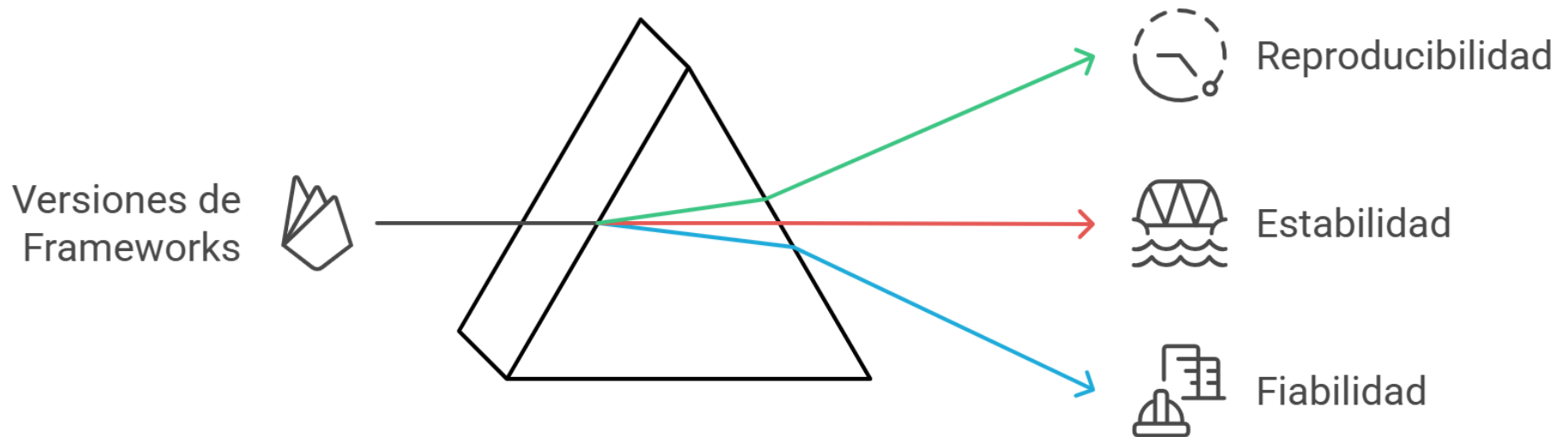
AUTOMATIZACIÓN Y ENTORNOS DE CONSTRUCCIÓN

A solid green horizontal bar at the bottom of the slide.

Ciclo de Gestión de Dependencias



Revelando el Impacto de las Versiones en las Pruebas



La gestión de dependencias
es un pilar de la calidad del
software.

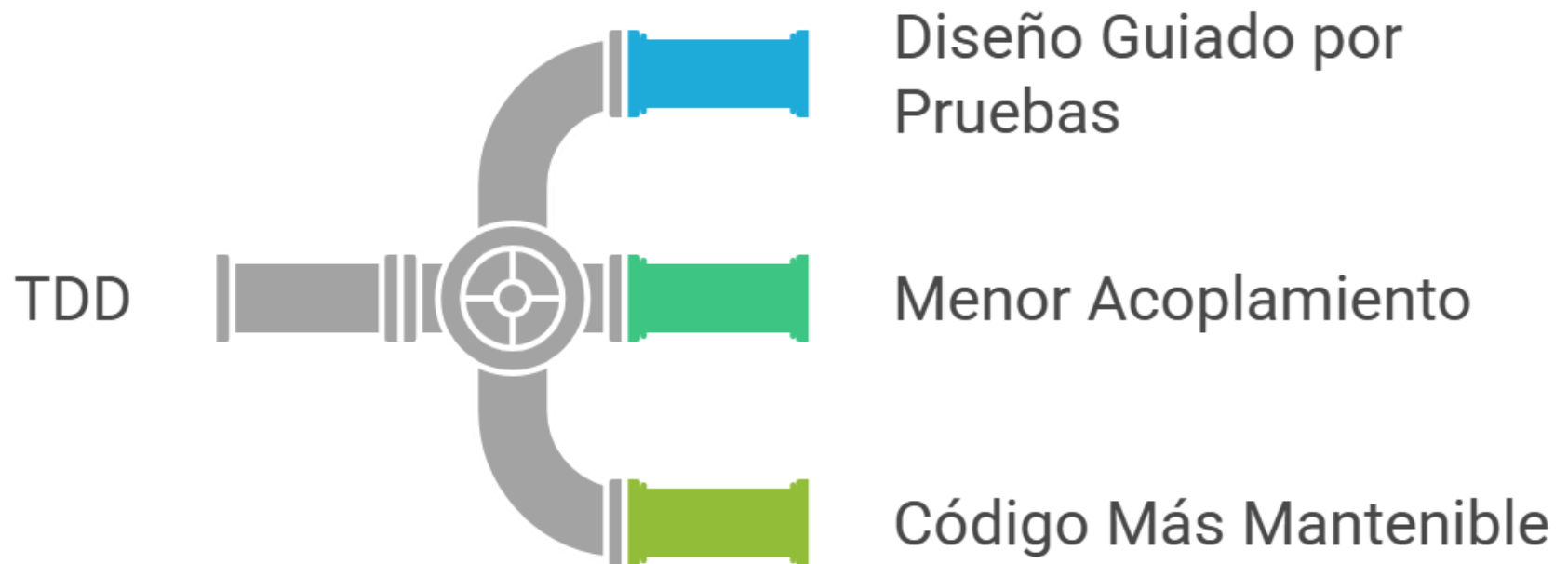
Introducción a TDD

DESARROLLO BASADO EN PRUEBAS — TDD

A solid green horizontal bar at the bottom of the slide.



Beneficios del TDD



TDD e integración con CI/CD

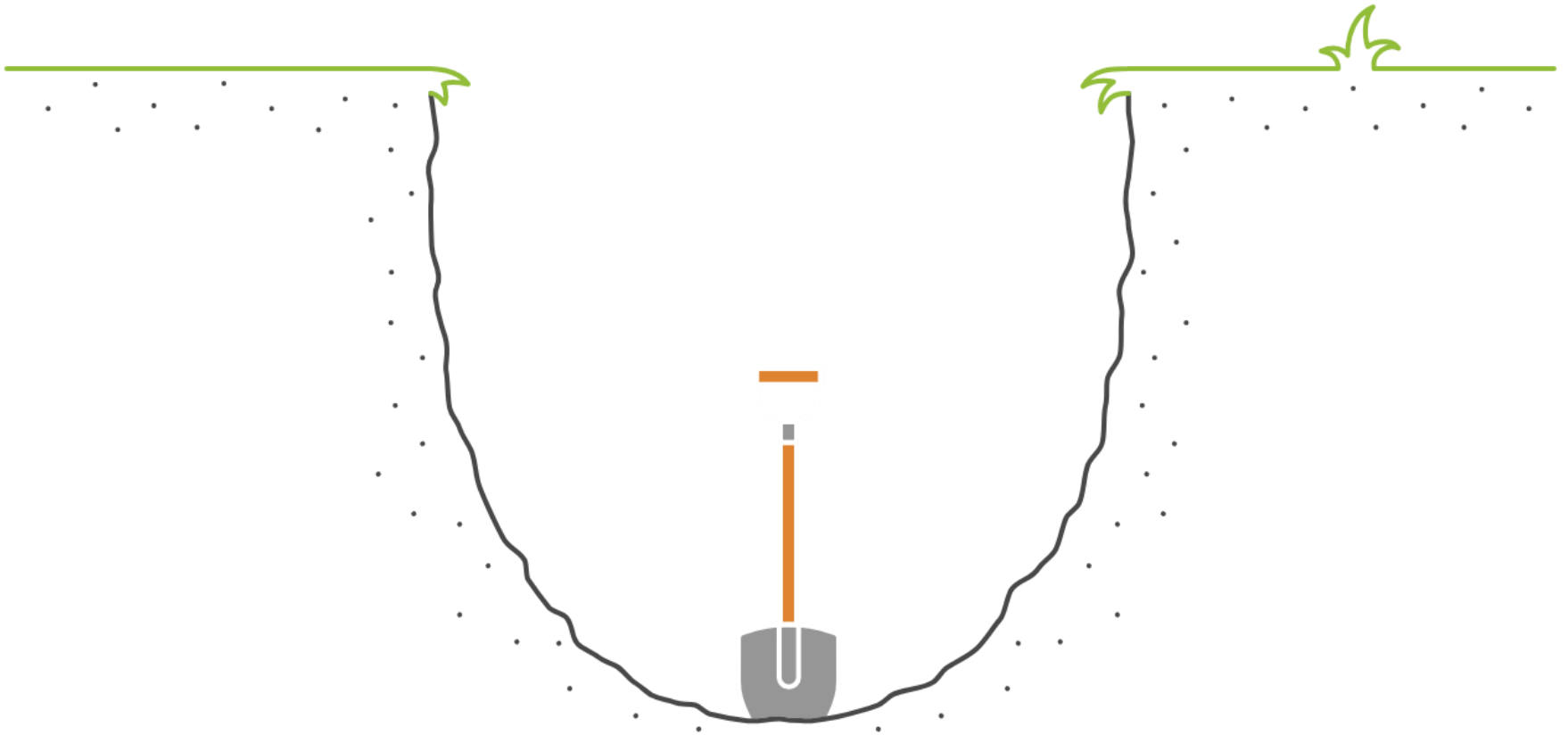
DESARROLLO BASADO EN PRUEBAS — TDD

A solid green horizontal bar at the bottom of the slide.

TDD e integración con CI/CD



Los fallos en las pruebas retrasan el despliegue a producción



TDD y control de versiones

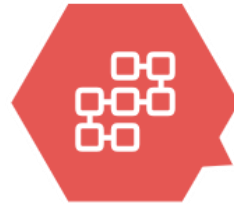
DESARROLLO BASADO EN PRUEBAS — TDD

A solid green horizontal bar at the bottom of the slide.

TDD y control de versiones

Historial trazable

Mantén un historial trazable de los cambios en el código y las pruebas.



Commits pequeños

Realiza commits pequeños y frecuentes para facilitar la revisión.



Pruebas versionadas

Versiona tus pruebas junto con el código para mantener la coherencia.

“Un cambio sin prueba es
un cambio incompleto”